# LAGUNTZArako WEB guneak

Thread - https://docs.Oracle.com/javase/8/docs/api/java/lang/Thread.html

**Runnable – https://docs.Oracle.com/javase/8/docs/api/java/lang/Runnable.html**

**Java.útil.concurrent** - https://docs.Oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html

**Concurrencia en Java** - https://docs.Oracle.com/javase/tutorial/essential/concurrency/index.html

**Java Concurrency** - https://www.baeldung.com/java-concurrency



# LANKETA – I – JOSEBA

## JAVA.UTIL.CONCURRENT

1. Executor
2. Queues
3. Sincronizadores
4. Estructura de datos concurrentes
5. Interfaces: ExecutorService, Callable, Future



# LANKETA – II

## PROGRAMACIÓN ASINCRONA

1. Interfaz Runnable
2. La clase Thread
3. El método Sleep
4. Interrupciones
5. Compartición de información

# LANKETA – III

Problemas y Soluciones de la programación concurrente. Conceptos y Sincronización

1. Recursos Compartidos
2. Dependencias
3. Condición de Bernstein
4. Acción y acceso atómicos.
5. Sección critica.
6. Thread Safety

# LANKETA – IV

Problemas de la programación concurrente.

1. Interbloqueo o deadlock
2. Muerte por inanición
3. Condiciones de carrera
4. Inconsistencia de memoria
5. Condiciones deslizadas

# LANKETA – V

Sincronización básica:

1. variables volatile.
2. Wait, notify y notifyAll
3. Join
4. Estructura de datos resistentes a hilos

# LANKETA –VI

Sincronización Avanzada:

1. Exclusión mutua,
2. Synchronize
3. Monitores
4. Semaforos

# LANKETA –VII – JOSEBA

## Actividades de aplicación:

1. Explica la diferencia entre programación secuencial y concurrente.
2. Explica que aporta crear un hilo a partir de la interfaz *Runnable* frente a hacerlo heredando de *Thread* desde el punto de visa de la herencia
3. Enumera y explica los mecanismos más importantes para compartir información entre hilos.
4. ¿Qué garantizan las variables de tipo *volatile*?
5. ¿Cuál es la utilidad de los semáforos en la programación concurrente?

